

UNIVERSIDAD DE MONTERREY

School of Engineering and Technologies

Course: Artificial Intelligence II

Final project: Sports Analytics using Deep Learning Models

Thursday May 21th, 2026

Lecturer: Dr. Andrés Hernández Gutiérrez

Renata García Morales

ID: 612194

Einer Barba Abdala

ID: 595839

“We hereby affirm that we have done this activity with academic integrity”

Index

Introduction to vision analytics and deep learning for object detection and classification	3
Problem statement	4
Methodology	5
References	7
Video Demo	8
Final github	8

Introduction to vision analytics and deep learning for object detection and classification

According to Universidad Europea (2025), when referring to sports analytics it is meant as the use of data, statistical methods, and computational techniques that are used for extraction of valued insights from sports performances and its purpose is to have a more objective understanding of the various elements of the field such as the movements of the players, the ball distribution and the patterns of activity throughout the match. This has value for coaches and athletes when it comes to evaluating the performance and identifying the strengths and the weaknesses so that new strategies can be defined when it comes to tactical decisions in the field or the training. In this field of sport analytics there has been a big growth because of deep learning which has become key. Through the use of convolutional neural networks, transfer learning and object detection frameworks now it is possible to automatically detect the previously mentioned elements in sports footage.

Talking about object detection this is centered in identifying the location and the class of objects in an image which for example could be the frame of a video. The output of an object detector is a set of bounding boxes enclosing the detected objects accompanied by the class labels and the confidence scores. This for example is different from image classification which only identifies what is present in an image by also specifying where its object is located. It could be said that object detection frameworks are divided into two broad categories that are the two stage detectors and single stage detectors. Two stage detectors like Faster R-CNN first generate a set of candidate regions and then classify each one individually leading to a great accuracy but at the cost of speed. On the other hand, single stage detectors like the YOLO family which is used during this project predict bounding boxes and class probabilities in a single forward pass through the network. This makes it faster and better suited for real time applications like the case focused in this project. (Ultralytics, n.d.)

Another important element for this project is transfer learning which is characterized by using a model trained on a large dataset which is then reused as the starting point for a different task that is related. For object detection, a common practice is to start with a model that is pretrained on a big benchmark like that of MS COCO containing more than 300,000 annotated images. Then Fine-tuning adapts the later layers of the network to the specific classes of the new task which makes the results better and more efficient than starting from zero. (Ultralytics, n.d.)

Problem statement

The problem addressed during this project is the automatic detection of soccer plates and the ball taken from an aerial video sequence that was recorded by a DJI Mavic Pro drone that recorded the complete soccer field of the UDEM Campus. The total length of the video is of 2 minutes and 8 seconds that are recorded at 30 frames per second which adds in total 3,846 frames with a resolution of 1280 x 720 px. The objective of this project is to develop a deep learning based object detection pipeline that is able to accurately capture the detection of the two object classes throughout the entire video. These classes are the ball and the player. The class player consists of any soccer player that is visible in the frame, this includes those who are partially seen. The other class will be the ball that represents a big challenge due to its small size in the aerial view.

The detections must include the count of players on each side of the field, the count of players on each side of the field and the position of the ball. The key challenges that were taken in consideration were the aerial perspective which reduces the size of the objects with players being small figures difficult to detect even by the human eye occupying only a few pixels. The second challenge are the shadows from trees and drone movement that although isn't as prominent it may still call visual noise that may provoke false positives or missed detections when a player passes near a tree. A third complication is a bird that crosses through some frames of the video which may also complicate the detection of the two classes. At last, computer efficiency and time limits need to be taken into account throughout the whole process since there are limitations when it comes to the deadline of the project. Some options may be better to produce a higher accuracy but a balance between speed and accuracy needs to be met.

Methodology

Based on the previous investigation, three architectures were evaluated in order to understand which option best suited this project. These three were YOLO26n, YOLOv8 and Faster R-CNN. The chosen option was YOLO26n which is the nano variant of the YOLO26 family that was developed by Ultralytics in January of 2026. This model was preferred because compared to Faster R-CNN it is faster, processing images in approximately 1.7 milliseconds on a GPU. When compared to YOLOv8, this newer version is lighter with 2.4 million parameters and has demonstrated a more stable localisation on small custom datasets because of its lower model capacity reducing the risk of overfitting. It also has an anchor-free design that makes it robust to the positional changes caused by the drone movement. The model uses fine-tuning, using pretrained weights from the MS COCO dataset. Applying transfer learning instead of training from nothing was considered as a good option because COCO includes the classes person and sports ball which are close to the target classes of the project.

At the start, the video was processed using OpenCv in order to extract a certain 700 frames from the video although the requirements asked for at least 575. This was done by using a fixed interval sampling where a frame was selected every 20 frames. There are other methods such as over-scene change detection but this was considered not suitable because the aerial view of the field doesn't produce scene changes. The frame extraction was divided into two phases because an initial model was implemented for the initial update of the project where only 189 frames were extracted. Then for the final version the last 405 frames were extracted making it a total of 511. In the first phase one frame was extracted every 20 frames throughout the entire video while in the second phase the rest of the frames were extracted by using a smaller interval of 4 frames but skipping any frame positions that were previously captured.

The next section was the data annotation where all the extracted frames were annotated using Roboflow as the annotation platform because of its compatibility with the YOLO annotation format. The bounding boxes were drawn as accurately as possible by drawing them tightly to the body of the players and the shadows were excluded, this was also applied to the ball. The two classes defined were 0 which corresponded to the player and class 1 that corresponded to the ball. All the coordinate values were normalised to the range of 0,1 relative to the image dimensions using the standard of the YOLO format in which each object is represented as a single line like class_id x_center y_center width height.

The dataset was then split into training, validation and test subsets using a 70, 15, 15 split that was done with the function of scikit learn. The split was done manually instead of using Roboflow in order to maintain full control over the distribution. The final split resulted in 492 images for training, 105 for validation and 106 for testing. Each image was copied with its annotation file into the adequate folder by using the shutil library. Then, the data.yaml file was updated so that it reflected the new folder paths.

During the training of the model, the YOLO26n model was fine-tuned on a custom dataset using the Ultralytics library and running locally on an NVIDIA GeForce RTX 2080 SUPER GPU. The image size was set to 1280 pixels which was done in order to improve the

detection of the ball since a higher resolution allows the model to see more detail in each frame. The training was run for a total of 100 epochs with a batch size of 8 which was the largest that fit within the available GPU memory at its resolution. In order to help the model learn to detect the ball more effectively, three loss weights were adjusted from their default values. The box loss weight was increased to 12.0, classification loss weight to 3.0 and distribution focal loss to 2.0. These values control how much each type of error is penalized in training and by increasing them it forces the model to pay more attention to getting bounding boxes positions and class assignments correctly which is essential for the ball. Another element was the optimizer which was set to automatic selection resulting in AdamW. Automatic Mixed Precision (AMP) was used in order to make the training faster by using lower numerical precision where it was possible without affecting the accuracy. The best weights were finally saved based on the highest validation mAP50 that was seen throughout the run.

In order to evaluate how well the model generalises to unseen data the best checkpoint was then tested against the test split, it being the set that was never seen during the training and the validation. Here are the results rounded to 3 decimals.

Class	Precision	Recall	mAP50	mAP50-95
All	0.930	0.909	0.935	0.546
Ball	0.890	0.846	0.896	0.471
Player	0.069	0.972	0.975	0.622

The player class achieved overall great results confirming that the model detects players correctly across unseen frames and the ball class although with not the same quality of results, they were still positive outcomes giving a solid performance. To continue the validation, it was tested on truly isolated samples, inference was run on 10 frames extracted at positions that were not included in any split. The ball was detected in 9 out of 10 frames, and players were detected in all 10 frames.

At the end, in order to do the field live video predictions the trained model was applied to the complete video in order to generate the visual analytics per frame. Each frame is processed through the detection model and the results are used to count players by zone and inform about the ball position. The Field zones were defined manually on the first frame, this was done by clicking polygon vertices for the full boundary of the field. For each of the frames, the model predicts the players and ball bounding boxes and each of these is classified into a zone based on where the centroid falls.

Personal Conclusions

Einer:

I found this project to be a true full measure of a lot of topics covered during this course. It was a practical, realistic application of the different concepts we learned about model training and object detection. Exploring the full pipeline of such a project, starting with an initial set of data (in this case, frames), to labeling, training, and validating, was a truly valuable learning experience overall. To be able to observe in real time the actual effects of our work, instead of focusing solely on, for example, an accuracy value as a benchmark of success, was quite remarkable. This project easily illustrated the usefulness of these kinds of object detection models. Data analysis is my career path of interest, due to which I found this pipeline project an excellent learning experience, and hope to be able to use it professionally in the future.

Renata:

It can be concluded that the project had good results when it came to detecting the players and the ball demonstrating that the selection of the YOLO family, specifically, the YOLO26n was adequate for the project. This was because of how effective it is when it comes to a balance between accuracy, speed and ease of implementation. One of the most challenging parts throughout the project was the annotation of the images which required a significant amount of time. Another detail was that this was my first time working with YOLO so I had to pass through the process of learning how to prepare the dataset, train the model and evaluate the performance. The process of the project wasn't completely smooth since during the first submission there were problems with the low confidence score when it came to the ball which happened because of its small size. The results improved when adding a bigger amount of frames however it was important to adjust the training parameters so that the model could perform better. At the end, it could be understood that a successful implementation of a model depends on various things such as the careful preparation of the data, experimentation and continuous improvements.

References

- GeeksforGeeks. (2025, January 23). Extract images from video in Python. GeeksforGeeks. <https://www.geeksforgeeks.org/python/extract-images-from-video-in-python/>
- Kandhare, M., & Gisselbrecht, T. (2024). An empirical comparison of video frame sampling methods for multi-modal RAG retrieval. arXiv. <https://arxiv.org/html/2408.03340v1>
- Roboflow. (2022, June 15). Tips for how to label images. Roboflow Blog. <https://blog.roboflow.com/tips-for-how-to-label-images/>
- Ultralytics. (n.d.). COCO dataset. Ultralytics Docs. <https://docs.ultralytics.com/datasets/detect/coco/>

Ultralytics. (n.d.). Integración con Roboflow. Ultralytics Docs.

<https://docs.ultralytics.com/es/integrations/roboflow/>

Ultralytics. (n.d.). Object detection. Ultralytics Docs. <https://docs.ultralytics.com/tasks/detect/>

Ultralytics. (n.d.). Roboflow collect. Ultralytics Docs.

<https://docs.ultralytics.com/es/integrations/roboflow/#roboflow-collect>

Ultralytics. (n.d.). Train mode. Ultralytics Docs. <https://docs.ultralytics.com/modes/train/>

Ultralytics. (n.d.). Ultralytics tutorial notebook. Google Colab.

<https://colab.research.google.com/github/ultralytics/ultralytics/blob/main/examples/tutorial.ipynb#scrollTo=zR9ZbuQCH7FX>

Universidad Europea. (2025, December 30). Sports analytics: qué es y cómo optimiza el rendimiento. <https://universidadeuropea.com/blog/sports-analytics/>

Video Demo

Link: <https://youtu.be/fBsgSCLFmyc>

Github Repository

Link: <https://github.com/einerbarba-u/final-project-AI-II.git>